

Звіт

до Лабораторної роботи №6

Інтерфейси та множинна реалізація інтерфейсів у Java

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Мета роботи

Метою лабораторної роботи є отримання практичних навичок використання інтерфейсів у Java, застосування інтерфейсів як поведінкових контрактів та демонстрація множинної реалізації інтерфейсів у конкретних класах.

У ході роботи необхідно створити щонайменше два інтерфейси, реалізувати кілька конкретних класів, показати клас, який реалізує більше ніж один інтерфейс, а також продемонструвати роботу програми через типи інтерфейсів.

2. Опис предметної області

У даній лабораторній роботі розглядається предметна область засобів командної роботи над проєктами.

У таких системах можуть існувати різні об'єкти: картки завдань, звіти про помилки та примітки до рев'ю. Частину таких об'єктів можна призначати виконавцю, а частину можна коментувати.

Саме тому для цієї моделі доцільно використовувати інтерфейси. Інтерфейси описують не те, чим є об'єкт, а те, що він може робити.

3. Використані інтерфейси

У роботі створено два інтерфейси:

- Assignable — об'єкт можна призначити виконавцю;
- Commentable — до об'єкта можна додати коментар.

3.1 Інтерфейс Assignable

Інтерфейс Assignable описує поведінку об'єктів, які можна призначити відповідальному виконавцю.

```
public interface Assignable {  
    String assignTo(String personName);  
  
    String getAssignee();  
}
```

Цей інтерфейс є змістовним, тому що у командній роботі завдання або звіт про помилку зазвичай мають відповідального виконавця.

3.2 Інтерфейс Commentable

Інтерфейс Commentable описує поведінку об'єктів, до яких можна додавати коментарі.

```
public interface Commentable {  
    String addComment(String comment);  
  
    int getCommentsCount();  
}
```

Цей інтерфейс також є змістовним, тому що у командній роботі важливо залишати коментарі до завдань, помилок або приміток.

4. Реалізовані класи

У роботі реалізовано три конкретні класи:

- TaskCard;
- BugReport;
- ReviewNote.

4.1 Клас TaskCard

Клас TaskCard описує картку завдання у системі командної роботи.

Поля класу:

- title — назва завдання;
- description — опис завдання;
- assignee — виконавець;
- commentsCount — кількість коментарів.

Клас реалізує два інтерфейси:

```
public class TaskCard implements Assignable, Commentable
```

Це логічно, тому що завдання можна призначити виконавцю і до нього можна додавати коментарі.

4.2 Клас BugReport

Клас BugReport описує звіт про помилку.

Поля класу:

- bugId — номер помилки;
- severity — рівень важливості;
- assignee — відповідальний виконавець;
- commentsCount — кількість коментарів.

Клас також реалізує два інтерфейси:

```
public class BugReport implements Assignable, Commentable
```

Це є коректним, тому що звіт про помилку можна призначити розробнику і до нього можна додавати коментарі.

4.3 Клас ReviewNote

Клас ReviewNote описує примітку до рев'ю коду.

Поля класу:

- author — автор примітки;
- text — текст примітки;
- commentsCount — кількість коментарів.

Клас реалізує тільки інтерфейс Commentable:

```
public class ReviewNote implements Commentable
```

Це логічно, тому що примітку до рев'ю можна коментувати, але її не обов'язково призначати виконавцю.

5. Множинна реалізація інтерфейсів

У роботі є класи, які реалізують більше ніж один інтерфейс.

Клас	Реалізовані інтерфейси	Пояснення
TaskCard	Assignable, Commentable	Завдання можна призначити виконавцю і коментувати.
BugReport	Assignable, Commentable	Помилку можна призначити відповідальному і коментувати.
ReviewNote	Commentable	Примітку до рев'ю можна коментувати.

Таким чином у програмі продемонстровано множинну реалізацію інтерфейсів у Java. Клас TaskCard і клас BugReport мають дві незалежні поведінкові ролі.

6. Використання інтерфейсів у клієнтському коді

Інтерфейсно-орієнтований клієнтський код показано у класі Main.

Спочатку створюються об'єкти конкретних класів:

```
TaskCard task = new TaskCard("Створити сторінку входу", "Додати просту форму авторизації");
```

```
BugReport bug = new BugReport("BUG-101", "Висока");
```

```
ReviewNote note = new ReviewNote("Артем", "Код потребує невеликих виправлень");
```

Далі об'єкти обробляються через інтерфейс Assignable:

```
Assignable[] assignableItems = {  
    task,  
    bug  
};  
  
for (Assignable item : assignableItems) {  
    System.out.println(item.assignTo("Колісниченко Артем"));  
}
```

У цьому фрагменті програма працює не напряму з класами TaskCard або BugReport, а через тип інтерфейсу Assignable.

Також об'єкти обробляються через інтерфейс Commentable:

```
Commentable[] commentableItems = {  
    task,  
    bug,  
    note  
};  
  
for (Commentable item : commentableItems) {  
    System.out.println(item.addComment("Перевірено куратором проекту"));  
}
```

У цьому фрагменті різні класи обробляються однаково, тому що всі вони реалізують інтерфейс Commentable.

7. Один об'єкт через два різні інтерфейси

У роботі показано один об'єкт через два різні інтерфейсні подання.

```
Assignable assignableView = task;  
Commentable commentableView = task;
```

```
System.out.println(assignableView.assignTo("Іван Петренко"));
System.out.println(commentableView.addComment("Завдання було
перепризначено"));
```

Об'єкт task має тип TaskCard, але його можна використовувати як Assignable і як Commentable. Це показує, що один клас може підтримувати кілька незалежних ролей.

8. Чому інтерфейси доречніші, ніж одна ієрархія класів

У цій роботі інтерфейси є кращим рішенням, ніж побудова однієї ієрархії класів, тому що Assignable і Commentable описують не тип об'єкта, а його можливості.

Наприклад, TaskCard і BugReport мають дві можливості: їх можна призначати виконавцю і коментувати. Клас ReviewNote має тільки одну з цих можливостей — його можна коментувати.

Ці класи не обов'язково повинні утворювати жорстку ієрархію успадкування. Натомість вони можуть реалізовувати спільні поведінкові контракти через інтерфейси.

9. UML-діаграма

Для створеної моделі було побудовано UML-діаграму. На ній показано інтерфейси, конкретні класи та зв'язки реалізації між ними.

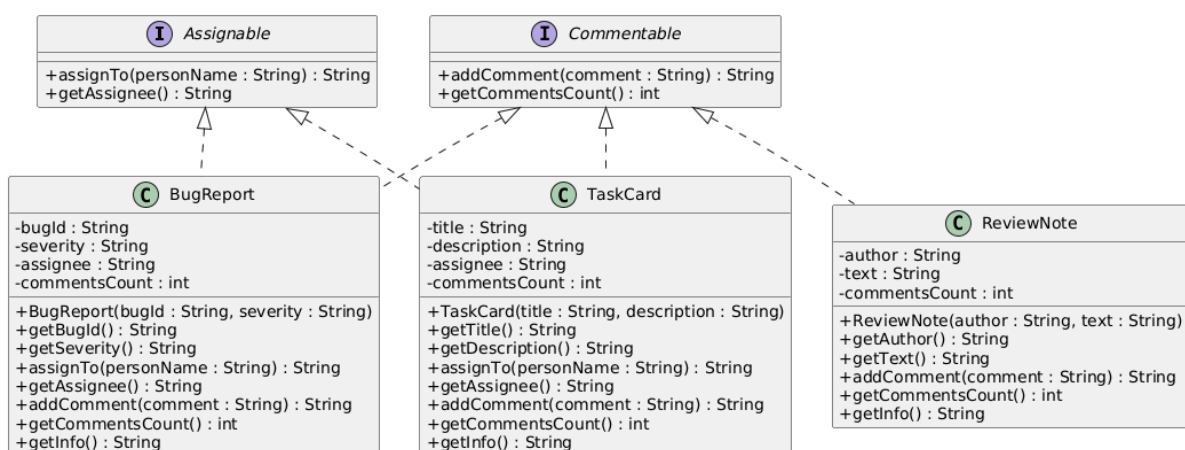


Рисунок 1 — UML-діаграма моделі з інтерфейсами

10. Демонстрація роботи програми

Після запуску класу Main програма демонструє обробку об'єктів через інтерфейси Assignable і Commentable, а також використання одного об'єкта через два різні інтерфейсні подання.

Приклад результату роботи:

Обробка об'єктів через інтерфейс Assignable

Завдання "Створити сторінку входу" призначено виконавцю: Колісниченко Артем

Помилку BUG-101 призначено виконавцю: Колісниченко Артем

Обробка об'єктів через інтерфейс Commentable

Коментар додано до завдання "Створити сторінку входу": Перевірено куратором проєкту

Коментар додано до помилки BUG-101: Перевірено куратором проєкту

Коментар додано до примітки від Артем: Перевірено куратором проєкту

Один об'єкт через два різні інтерфейси

Завдання "Створити сторінку входу" призначено виконавцю: Іван Петренко

Коментар додано до завдання "Створити сторінку входу": Завдання було перепризначено

```

Обробка об'єктів через інтерфейс Assignable
Task "Створити сторінку входу" assigned to Колісниченко Артем
Bug BUG-101 assigned to Колісниченко Артем

Обробка об'єктів через інтерфейс Commentable
Comment added to task "Створити сторінку входу": Перевірено куратором проєкту
Comment added to bug BUG-101: Перевірено куратором проєкту
Comment added to review note by Артем: Перевірено куратором проєкту

Один об'єкт через два різні інтерфейси
Task "Створити сторінку входу" assigned to Іван Петренко
Comment added to task "Створити сторінку входу": Завдання було перепризначено

Підсумкова інформація
Task: Створити сторінку входу, assignee: Іван Петренко, comments: 2
Bug: BUG-101, severity: Висока, assignee: Колісниченко Артем, comments: 1
Review note by Артем, comments: 1

Process finished with exit code 0

```

Рисунок 2 — результат запуску програми

11. Тестування

Для перевірки роботи програми були створені тести JUnit Jupiter.

Тестові класи:

- AssignableContractTest;
- CommentableContractTest;
- TaskCardTest;
- BugReportTest;
- ReviewNoteTest.

Тести перевіряють роботу контрактів інтерфейсів, створення об'єктів, збереження полів і використання одного об'єкта через два різні інтерфейси.

```

[INFO] Running ua.khpi.oop.lab07.contracts.AssignableContractTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.052 s -- in ua.khpi.oop.lab07.contracts.AssignableContractTest
[INFO] Running ua.khpi.oop.lab07.contracts.CommentableContractTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s -- in ua.khpi.oop.lab07.contracts.CommentableContractTest
[INFO] Running ua.khpi.oop.lab07.model.BugReportTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.004 s -- in ua.khpi.oop.lab07.model.BugReportTest
[INFO] Running ua.khpi.oop.lab07.model.ReviewNoteTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.004 s -- in ua.khpi.oop.lab07.model.ReviewNoteTest
[INFO] Running ua.khpi.oop.lab07.model.TaskCardTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.004 s -- in ua.khpi.oop.lab07.model.TaskCardTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0

```

Рисунок 3 — результат виконання тестів

12. Додаткове завдання: аналіз попередньої моделі

У попередніх лабораторних роботах розглядалася модель спортивного клубу. У ній були класи SportsClub, Student, Coach, SportSection, Schedule та Enrollment.

У цій моделі можна використати інтерфейси для опису окремих можливостей. Наприклад, можна виділити інтерфейс Describable, який описує можливість отримати короткий опис об'єкта.

```
public interface Describable {  
    String getDescription();  
}
```

Такий інтерфейс могли б реалізовувати класи:

- Student;
- Coach;
- SportSection;
- Enrollment.

Також можна було б виділити інтерфейс Schedulable, який описує об'єкти, що мають розклад.

```
public interface Schedulable {  
    String getScheduleInfo();  
}
```

Його міг би реалізовувати клас SportSection, тому що спортивна секція має розклад занять.

Таке перепроекування може покращити модель, тому що дозволяє працювати з об'єктами за їхніми можливостями, а не створювати штучне успадкування між несхожими класами.

13. Висновок

У ході виконання лабораторної роботи було створено модель засобів командної роботи над проєктами.

Було реалізовано два інтерфейси: Assignable і Commentable. Також було створено три конкретні класи: TaskCard, BugReport і ReviewNote.

Класи TaskCard і BugReport реалізують два інтерфейси, що демонструє множинну реалізацію інтерфейсів у Java.

У класі Main було показано інтерфейсно-орієнтований клієнтський код: об'єкти обробляються через типи Assignable і Commentable.

Таке рішення є зручним, тому що інтерфейси описують незалежні поведінкові ролі об'єктів і не змушують будувати штучну ієрархію класів.